Informe: Motor de búsqueda de funciones similares vía AST

Lenguajes Formales y Autómatas, 2026 · **Integrantes:** Verón Neremias, Franchino Tomas

1. Decisiones de diseño

Dividimos la herramienta en cuatro módulos que se usan en cadena: `extraccion.py` → `normalizacion.py` → `serializacion.py` → `similitud.py`, y `buscar_similares.py` los junta en la CLI. Como ejemplo usamos `def sumar(a, b): return a + b`.

Extracción. Recorremos la carpeta con `Path.rglob("*.py")` y parseamos cada archivo con `ast.parse()`. Si uno tiene un error de sintaxis lo salteamos con un aviso para no cortar el análisis. De cada `FunctionDef` y `AsyncFunctionDef` guardamos nombre, archivo y subárbol. Al trabajar con el AST, los comentarios y espacios quedan afuera solos.

Normalización. Cada función tiene su propio diccionario que cambia parámetros y variables asignadas por `Var1`, `Var2`, ... en el orden en que aparecen, así `sumar(a, b)` y `sumar(x, y)` quedan iguales. Los nombres que no se definen en la función (`len`, `print`, `range`) quedan como están, porque usar `len` no es lo mismo que usar `sum`. En los literales nos apartamos un poco del enunciado: en vez de un solo `LIT` usamos `LIT_NUM`, `LIT_STR` y `LIT_BOOL`, para que `total = 0` y `total = 100` sigan dando 100% pero `x + 1` (suma) no se confunda con `x + "a"` (concatenación).

Serialización. No usamos `ast.walk()` porque recorre por niveles y no en pre-order, así que hicimos una función recursiva `preorder()` que agrega el nodo y después sus hijos de izquierda a derecha. Se emite el tipo de cada nodo (`For`, `If`, `BinOp`, `Add`...) o el nombre normalizado si es una variable. Sacamos `Load` y `Store` porque aparecen después de cada variable y solo agregan ruido. El nombre de la función nunca se emite. El ejemplo queda así: `FunctionDef, arguments, Var1, Var2, Return, BinOp, Var1, Add, Var2`.

2. Métrica elegida y por qué

Implementamos las dos métricas del enunciado y el puntaje final es el promedio de ambas (`similitud_hibrida`):

- **Jaccard sobre trigramas:** cortamos la secuencia en grupos de 3 tokens seguidos y dividimos los grupos en común por el total de grupos distintos. Es exigente: separa bien funciones que solo se parecen "de forma", pero un solo token distinto rompe hasta 3 trigramas.
- **Levenshtein normalizada:** cuántos tokens hay que insertar, borrar o cambiar para igualar las secuencias, dividido por el largo de la más larga (`1 - distancia / largo`). Tolera los cambios chicos, pero es generosa con funciones distintas de largo parecido, porque casi todas comparten el esqueleto (`FunctionDef`, `arguments`, `Return`).

Por ejemplo, `sumar(lista)` contra `invertir(texto)` (mismo esqueleto, distinta tarea) da 70,6% con Levenshtein, que sola ya la marcaría como clon; Jaccard le da 17,4%. Y al intercambiar dos líneas de `estadisticas` pasa al revés: Jaccard baja a 78,3% y Levenshtein queda en 90,7%. Promediando, una compensa a la otra. Para elegir n probamos con 2, 3 y 4 ("Final" usa n = 3):

Caso	J n=2	J n=3	J n=4	Lev.	Final	¿Clon?
Variables renombradas (`total` / `calcular`)	100%	100%	100%	100%	100%	Sí

Caso	J n=2	J n=3	J n=4	Lev.	Final	¿Clon?
Dos líneas del `for` intercambiadas (`estadisticas`)	90,5%	78,3%	66,7%	90,7%	84,5%	Sí
Un `print` agregado en `estadisticas`	86,7%	83,0%	78,7%	91,5%	87,2%	Sí
Mismo esqueleto, otra tarea (`sumar` / `invertir`)	33,3%	17,4%	8,3%	70,6%	44,0%	No
Suma de números / parseo de texto	10,3%	7,1%	4,9%	26,5%	16,8%	No
Iterativa / recursiva (`sumar` / `fact`)	6,9%	3,3%	0,0%	25,0%	14,2%	No

Con n = 2 los pares son muy genéricos (`Return, Var2`, "devolver una variable", aparece en muchísimas funciones) y las funciones distintas se acercan: `sumar` / `invertir` sube de 17,4% a 33,3%. Con n = 4 intercambiar dos líneas ya deja Jaccard en 66,7%. Con n = 3 quedó la mayor distancia entre el peor clon y la función distinta más parecida: 40,5 puntos, contra 37,1 con n = 2 y 39,2 con n = 4.

Umbral (`UMBRAL_SIMILITUD = 0.70`). Los clones quedaron en 84,5% o más y las funciones distintas en 46% o menos (`sumar` / `contar_pares` da 45,8%), salvo funciones muy cortas. No lo pusimos en la mitad (~65%) para dejar más margen contra falsos positivos, porque las funciones que "recorren una lista y acumulan" son muy comunes.

3. Limitaciones conocidas

Clones tipo 4 (hacen lo mismo, pero escrito distinto). La herramienta compara la forma del árbol, no lo que devuelve la función. Sumar una lista con un `for` genera `Assign, For, AugAssign, Add...`, mientras que `return sum(lista)` genera solo `Return, Call, sum, Var1`. Dan el mismo resultado, pero fuera del encabezado (`FunctionDef, arguments, Var1`) casi no comparten tokens: 16,3%. Un `for` contra un `while` que hacen la misma suma da 34,1%.

Funciones cortas. Con pocos tokens, cada uno pesa mucho. `return a + b` y `return a - b` generan 9 tokens y solo cambia `Add` por `Sub`, así que comparten 8 de 9 y dan 72,2%: se marcan como duplicadas aunque una suma y la otra resta. Al revés, agregar un `print` en `sumar(lista)` (15 tokens) suma 4 tokens nuevos, la baja a 68,4% y deja de detectarse; el mismo `print` en `estadisticas` (43 tokens) da 87,2%.

Orden de las variables. Las variables se numeran en el orden en que aparecen. Si en `resumen(d)` escribimos `a = len(d)` y después `b = sum(d)`, `a` es `Var2` y `b` es `Var3`; si invertimos esas dos líneas, los números se cruzan. Ya no coinciden ni las asignaciones (`Var2 = len(...)` contra `Var2 = sum(...)`) ni el `return b / a` (`Var3 / Var2` contra `Var2 / Var3`), y siendo la misma función da 58,5%. En el test de líneas reordenadas no pasa porque esas variables ya estaban numeradas antes.

4. Posibles mejoras

Ordenar los operandos de `+` y `*`. Hoy los tokens se comparan en el orden en que aparecen: `return a + b` queda `BinOp, Var1, Add, Var2` y `return b + a` queda `BinOp, Var2, Add, Var1`. Hacen lo mismo, pero dan 58,9% y no se detectan. La mejora es que, después de normalizar, en cada suma o multiplicación se ponga siempre primero el mismo operando (por ejemplo, ordenándolos alfabéticamente). Así las dos generan la misma secuencia y darían 100%. No se aplica a `-` ni a `/`, porque `a - b` no es lo mismo que `b - a`.

Ignorar el docstring. Los comentarios con `#` no llegan al AST, pero el docstring (`"""..."""`) sí, porque para Python es un string dentro del cuerpo. Documentar una función le agrega dos tokens (`Expr, LIT_STR`) que no cambian lo que hace, y en una función corta eso pesa mucho: `sumar(a, b)` con y sin docstring da 63,6% y no se detecta. Si antes de normalizar descartamos la primera línea del cuerpo cuando es un string suelto, las dos versiones quedan con los mismos 9 tokens y darían 100%.